

Name: Huddle - huddle fits because it represents a group of people getting together to organise something. it also sounds modern kind of.

Goal:

To make it easier for friend groups to organize and commit to social plans by providing a simple platform where users can propose activities, vote on them, and confirm attendance without relying on messy group chats or inconsistent responses.

Context:

Friend groups often struggle to coordinate plans due to:

- Low engagements in group chats (ignored messages or slowed replies)
- Indecision when choosing activities
- Difficulty finding times that suit everybody
- plans falling through due to lack of commitment

This can lead to immense frustration - especially if you are the one trying to plan something - and fewer group activities. The app will be designed for informal social groups such as friends, classmates, or clubs who want a faster and clearer way to make group decisions.

Objectives:

- Allows user to login to create and/or join friend groups
- Enables users to propose activities or events
- Lets group members vote on proposed plans (e.g. yes/no/maybe or ranked voting)
- Shows clear outcomes based on group preferences
- Improve commitment by tracking who has agreed to attend
- reduce confusion and repeated messaging in chats

Scope:

Platform & System Overview:

- progressive Web App (PWA) that is accessible through web browsers
- connects to a backend server for processing user data, events, and voting
- uses an SQL database system to store users, groups, and event information

In Scope - Core Features:

- user account creation and login with google authenticator 2FA
- ability to create or join friend groups via invite code
- users can propose events with details (time, location, description)
- voting system for group members (yes/no/maybe)
- display of the most popular or confirmed event
- Attendance tracking to see who is committed

Out of Scope - to keep project manageable:

- real time chat or messaging systems
- advanced recommendation algorithms
- social media-like features (likes, comments, posts)
- Payment systems or ticket booking
- calendar syncing or complex scheduling tools

Hardware requirements:

- Smartphones/Laptop/Desktop (primary device)
- Internet connection for data syncing
- server hosting environment (cloud or local)

Software Interfaces:

- web browsers for user interaction
- backend server for logic and data handling
- database system for storage
- authentication application (google authenticator)
- optional: push notifications for updates and reminders

Ongoing Support:

- backend and database maintenance (bug fixes, updates)
- security updates to protect user data
- user feedback integration and feature improvements

Stakeholders:

- Primary users: Friend groups - students, young adults, clubs
- Secondary users: Event organizers within groups (the person proposing the idea/group leader)
- Developer/client: Me with a group of friends (and mr jones as evaluator)
- Indirect stakeholders: Schools or communities if its used in group settings such as after school clubs.

Boundaries

Operational & time

- The system must be completed within the school term, meaning high complexity features are outside the boundary
- While designed for friend groups, the system may face performance boundaries if a single 'group' exceeds a certain size (e.g. 100+ members voting together)

Operating system & software boundaries

- The system will rely heavily on google authenticator. If that device is down or a user loses access to their 2FA device/email, they are locked out of their Huddle.
- Unlike native apps, Huddle cannot access deep OS features like accessing contact lists or hardware-level alarm systems.
- The app is limited by the capabilities of modern browsers (Chrome, Safari, Firefox). Features may behave differently on iOS vs. Android

Hardware boundaries

- The system must remain functional over varying screen size: Smartphone to PC
- As a PWA, the app must be lightweight enough to run on old tech without lag
- The system requires an active internet connection to sync votes and event data

Security & data boundaries

- The system is bound by the structure of an SQL database. It cannot handle unstructured data (social media posts) outside of the predefined event fields.
- Information is bound by "Friend Groups" - users in group A cannot see events or participations in group B unless they have a specific invite code

Requirements

Functional Requirements	Specifications	Must/Nice
Key voting/counting feature that allows users to select a preference on a proposed plan	System must tally “Yes”, “No”, “Maybe” votes; display real time percentage of a groups consensus.	Must
Event Proposal system allowing users to input event details	Input fields for Event Name (max 30 chars); Date/Time picker; Location (max 100 chars)	Must
User authentication via Google Authenticator 2FA for secure login **potential google account login**	Requires a 6-digit time-based code after password entry	Must
Group creation/joining via a unique alphanumeric invite code	Generates a 6-digit random code; database must link new UID to the specific group id.	Must
Attendance Tracker to display a list of committed members - The person who put forth the activity can take attendance afterwards to confirm who attended and who didn't - After the date of activity, a notification can pop up asking them if they attended or not	Visual list “Confirmed” users; update status immediately upon “yes” or “no” vote	Must
A feature where you can assign roles to the attending members such as “food bringer” or “ride home” to let others know who is bringing what or how people are getting home.	Input bar accepts text input; minimum 3 characters; maximum 50 characters;	Nice
Ability to save popular roles such as “ride home” or “bringing snacks” depending on popularity within the application	Drop down menu next to the search bar that holds pre-selectable answers; must be approved by dev to reduce profanities;	Nice
Interactive event map that pings the location of the hangout	Integration with map python library; display a single marker (ping) based on the event's address string.	Nice

Non-Functional Requirements	Specifications	Must/Nice
Availability	The PWA must be accessible to users 99.5% of the time during peak social hours (9 a.m. - 10 p.m.)	Must
Responsiveness	The voting tally must update and be visible to all group members within 2 seconds of a vote being cast	Must
Data Security	All users and group information must be encrypted	Must
Compatibility	The interface must be able to render correctly on screens ranging from 320px (mobile) to 1920px (desktop)	Must
Usability	A first-time user must be able to create a group and propose an event in under 3 minutes without a tutorial	Nice
Scalability	The backend server and SQL database must support up to 50 concurrent users/group members without lag	Nice

Implementation

I have chosen the **Direct Cutover Approach** because Huddle is a new system that does not have a direct digital predecessor to run alongside. To manage the risks of a direct launch, I will be conducting extensive beta testing with primary stakeholders/clients throughout the development. This ensures that when the 'cutover' occurs, the software will already be proven to meet the user's needs.

Finance plan

Huddle is a zero-budget project that uses the "Freemium" ecosystem of modern web development to stay free. By utilising the free tiers of GitHub, the app stays within usage and budget limits for the school term. The only real investment is time, making it a financially sustainable and risk-free solution for this assignment.

SWOT Analysis

Strengths:

- It solves a painful point that is highly relatable to my target audience
- By explicitly excluding things like real time chats and payment systems, the software engineering project stays manageable for a student project.
- By using a PWA it ensures cross-platform compatibility (iOS, Android, laptop) without the complexity of separate app store submissions.
- Familiarity with Secure PWA Architecture significantly reduces the research needed in the development phase.

Weaknesses:

- The app is only useful if an entire friend group joins; getting one person to switch is easy but getting 5 – 10 people is hard.
- Relying on google authenticator adds security but also creates a barrier for entry. If a simple-minded user doesn't have the app or finds the 2FA process too "heavy" for a casual social plan, they may abandon the sign-up.
- Since this relies heavily on a central SQL database and backend server, any downtime or hosting issues completely breaks the Huddle web app for all users

Opportunities:

- Could potentially be used by corporate companies trying to find a modern and fun solution to organising workflows or assign fun team building tasks towards a team.
- Beyond friend groups, this is perfect for school clubs or sports teams that need to organize attendance tracking but don't need a full management suite.
- Successfully deploying a functional PWA with a backend and SQL database is a massive real-world achievement for my portfolio.
- If the core voting aspect is a success and is enjoyed by the community, implementations such as "Quick Polls" can be implemented for things other than events, e.g. "What should we eat?" or "Who is taking the train?"

Threats:

- People are heavily addicted to the likes of WhatsApp, Discord, Instagram, or iMessage. Moving them to a separate app for potentially only one specific task is a big hurdle.
- Introducing something new into a group of people who are already used to a different application might turn the app into a 'one-hit-wonder' where members think the gimmick is cool and useful but forget about it and move back to what they were using before quickly.

- Without a native apps push notification, users might forget to check the app to vote, leading back to one of the original problem of “Ignored messages”

Development Approach: Hybrid (WAgile)

For the development of Huddle, I have chosen a Hybrid/WAgile approach methodology. This approach combines the structured, upfront planning of the Waterfall model with the iterative, flexible execution of the Agile model.

Application to Huddle

The project will begin with a Waterfall-style planning phase. Because this project is a school project with a hard deadline (end of term), I must define the core requirements – such as SQL database and Google Authenticator integration – before any coding begins. This ensures the boundaries of the project are respected and the scope doesn’t creep into out-of-scope areas.

Once the requirements are set, I can then shift the project to Agile sprints where development can be broken down into cycles.

- Sprint 1: login, User Authentication, and group creation
- Sprint 2: Event proposal and voting logic
- Sprint 3: PWA responsiveness and UI

Justification

The client and stakeholders: Since the client includes myself, friends, and an evaluator (Mr. Jones), the hybrid model provides the best of both worlds. The Waterfall structure side gives the evaluator a clear roadmap and set of deliverables, whilst the Agile structure side allows me to test the system with actual clients/friends and tweak things like UI or UX based on their feedback

Timeline: The school term is a short, fixed window. Pure waterfall is too risky because if a bug is found too late, there will be no time to restart. Likewise, pure Agile is too loose,

as I might spend too much time on minor features and miss the final deadline. WAgile ensures the most important features are built first.

Project type (PWA): Developing a PWA across different browser types or screen sizes requires constant testing. Agile sprints allow me to catch layout issues on mobile early, rather than waiting till the very end of the project.

Comparison of alternative approaches

If I used Pure Waterfall: I would spend weeks writing documentation and only start coding at the end. If the 2FA system failed or the SQL database didn't sync correctly during the final week, the project would fail because there would be no time to go back and fix the design.

If I used Pure Agile: The project might lack a clear "end point". I might get distracted by adding social features such as "likes" - which are out of scope - and realise too late that I haven't finished the essential voting system required for the project.

GANTT chart + sprint plan

Phase/Sprint	Dates	Goal	Deliverable
Phase 0	Weeks 1 - 2	Ideation	3 Ideas, Huddle selected, diary + systems report started.
Phase 1	Weeks 3	Requirements	Client meeting, FR/NFR documented, SWOT done, boundaries established
Phase 2	Weeks 3 - 4	Research & planning	Hybrid process documented, GANTT + Sprint chart, social and ethical issues, system models (DFDs, structure charts, wireframes, etc.) + UAT plan
Phase 3	Weeks 4 -	Producing & implementing	Development sprints completed, client feedback for each sprint, difficulties and resolutions, all modules completed and integrated together
Phase 4	Weeks	Testing & evaluating	Create test data, UAT testing, documented bugs, optimise code, client sign off, compare feedback to functional requirements, write README

Sprint 1	Weeks 5	Database + skeleton	Working login page + user authentication, empty menu pages
Sprint 2	Weeks 6 – 7	Core features	Core event proposal and voting logic, group creation
Sprint 3	Weeks 7 - 8	Finalisation	PWA responsiveness & UI + UX design, front home page and UI complete
Sprint 4	Weeks 9	Documentation	Finalise systems report + Client log + Developer diary

Social & Ethical Issues

Privacy – The ethics of data collection

Many social apps over-collect data (e.g. location, contacts, or browsing habits), which can lead to surveillance creep where users feel tracked.

To address this issue, Huddle only collects the bare minimum data for function: an email address and a display name. Unlike other native apps, the PWA structure means the app cannot access deep OS features like user's contact list. This will create a privacy-first environment where friends can coordinate without the fear of their personal data being harvested or sold. Furthermore, the SQL database boundaries ensure the data is siloed: What happens in "friend group A" is entirely invisible to "friend group B", protecting the private social habits of different circles.

Security – Responsibility for user safety

A data breach or account takeover doesn't just leak the data; it can lead to physical safety risks as well (e.g., data breacher knowing exactly when and where a group of students will be meeting).

As a developer, I have an ethical responsibility to secure user coordination. To mitigate the risks of unauthorized access, Huddle will be integrated with Google Authentication Two-Factor-Authentication (2FA). Even if a user's password is leaked, the 2FA requirement acts as a critical secondary barrier. Additionally, the invite code system acts as a security boundary. Because there is no 'public search' for groups, a malicious actor cannot find or join a huddle without a direct human-sent link.

Accessibility – Inclusivity in the software design

Many software projects often exclude users by ignoring diverse physical or technological limitations, preventing individuals with disabilities from navigating their platforms.

Huddle addresses this by continuously adhering to the global Web Content Accessibility Guidelines (WCAG) accessibility standards. The PWA layout will scale dynamically across smartphones, laptops, and desktop computers. This responsive design ensures high quality readability and clear navigation on all screen sizes. Furthermore, the lightweight code runs smoothly on older hardware without causing performance lag. This prevents economic barriers from restricting user participation and social planning.

Environmental – reducing digital carbon footprint

Inefficient software and bloated cloud applications generate massive carbon emissions by consuming excessive electricity from server hosting. Bloated software increases server processing workloads, which drives up electrical consumptions and cooling needs.

Huddle addresses this environmental concern through its architectural and scope boundaries: by selecting a structured SQL database over unstructured alternatives, the system minimises data processing overhead. The decision to exclude high-complexity features such as social-media feeds and real-time chat reduces continuous server

hosting. This lightweight approach limits the energy required for hosting and lowers the project's overall digital carbon footprint.

Intellectual property – Ethical use of shared code

Unauthorised use of open-sourced code violates copyright laws and exploits community labour, potentially creating severe legal liabilities for software projects. Developers face an ethical obligation to respect the intellectual properties of others by software license.

Huddle utilises open-sourced components for its users interface layout, backend server framework, and SQL database connectors. The project maintains an inventory of third-party libraries and complies with licencing terms

Compliance and Legislative requirements

Privacy act 1988

Relevance:

- The app collects personalised identifiable data information from its users, including names, emails, and group codes
- The system stores the user-submitted text data such as specific event times, locations, and descriptions

Design address:

- User authentication is strictly managed using Google Authenticator Two-Factor Authentication (2FA) to prevent unauthorised account access
- The SQL database isolates data using unique group keys so 'Group A' users cannot view 'Group B' data without an explicit invite code

Copyright act 1968

Relevance:

- The PWA relies on external software libraries, frameworks, and interfaces to run in browsers
- User-generated fields allow people to input text content or descriptions that could potentially copy existing event material.

Design address:

- The system will use open-sourced software packages under verified licences such as Bootstrap

Quality Assurance Criteria

Criteria	What it means	How it will be measured
Functionality	All core features work correctly	Test whether user can login, join groups, propose events, vote, and confirm attendance successfully during UAT
Usability	The system is easy to use without help	A first-time user should be able to create or join a group and propose an event without a tutorial
Reliability	The system works consistently without crashing	Test with repeated login, voting, and event creation actions to check that no errors or crashes occur
Performance	The system responds quickly	Vote results and attendance updates should appear within 2 seconds of being submitted
Security	User data is protected from unauthorised access	Check that passwords are hashed, 2FA is required, and users cannot access other groups without the correct invite code
Compatibility	The app works across different devices	Test the interface on mobile, tablet, and desktop screens to ensure it displays correctly
Maintainability	The code is organised and easy to update	The project should include clear comments, modular code, and a README explaining setup and usage

UAT Plan

for

Huddle

Friend Group Social Planning PWA

Draft – Phase 2 | Version 1.0

1. Scope

1.1 Objectives and Business Requirements

The goal of this User Acceptance Test is to verify that Huddle - a Progressive Web App (PWA) for friend group social planning - meets its defined functional and non-functional requirements before launch.

Specifically, UAT will confirm that:

Objective	How Success Will Be Measured
Core features function correctly	Users can log in, create/join groups, propose events, vote, and track attendance without errors.
Performance meets targets	Vote tallies update within 2 seconds of being cast.
Security boundaries are enforced	Users cannot access other groups' data. 2FA blocks unauthorised logins.
Usability is sufficient	A first-time user can create a group and propose an event in under 3 minutes.
Compatibility across devices	The interface renders correctly on screens from 320px (mobile) to 1920px (desktop) and across Chrome, Firefox, and Safari.

1.2 Scope

Huddle is a new PWA designed to help friend groups organise and commit to social plans through event proposals, group voting, and attendance tracking. It replaces the frustration of unresponsive group chats with a structured, clear decision-making interface.

For this UAT, we will be testing:

- User registration and Google Authenticator 2FA login
- Group creation and joining via unique invite codes
- Event proposal form (name, date/time, location)
- Yes/No/Maybe voting system with real-time tally display
- Attendance tracking and confirmed member list
- Data isolation between separate friend groups (security)
- Responsive layout on mobile (320px), tablet, and desktop (1920px)
- Cross-browser compatibility (Chrome, Firefox, Safari)
- First-time usability (event creation within 3 minutes)

We are NOT testing in this UAT:

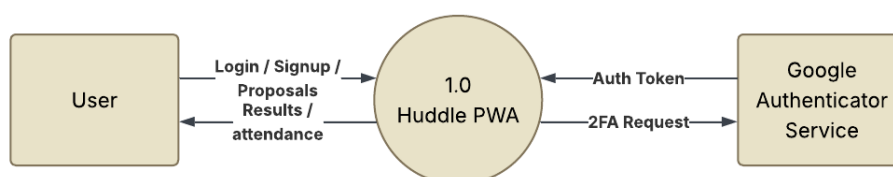
- Real-time chat or messaging features (out of scope for Huddle entirely)
- Payment systems, ticket booking, or calendar syncing
- Social media-style features (likes, comments, feeds)
- Advanced recommendation algorithms
- Performance under extreme load (50+ simultaneous users) — covered in separate load testing

1.3 System Diagrams

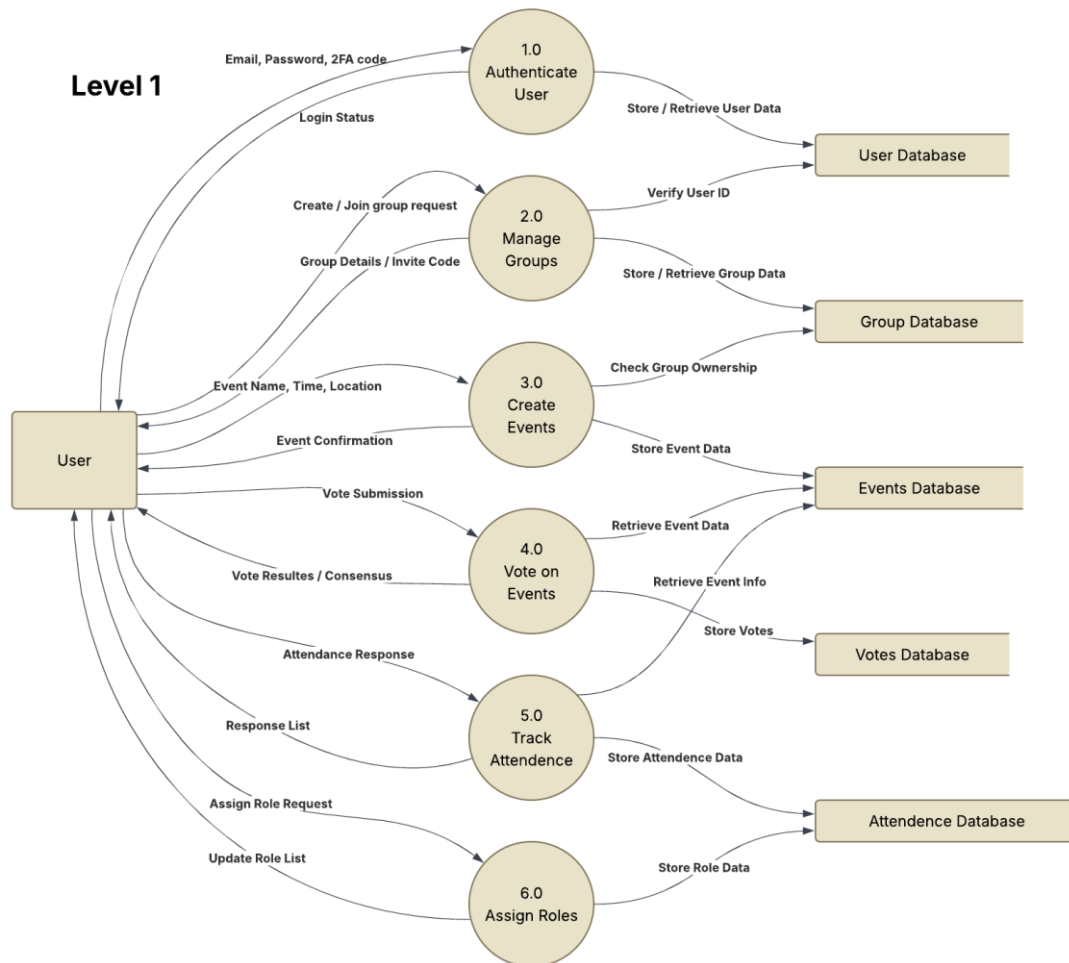
The diagrams below represent key structural and data-flow relationships within Huddle. UAT testers should review these to understand how the system components interact.

Level 0 DFD: Shows the overall data flow between the user, the Huddle system, and the authenticator.

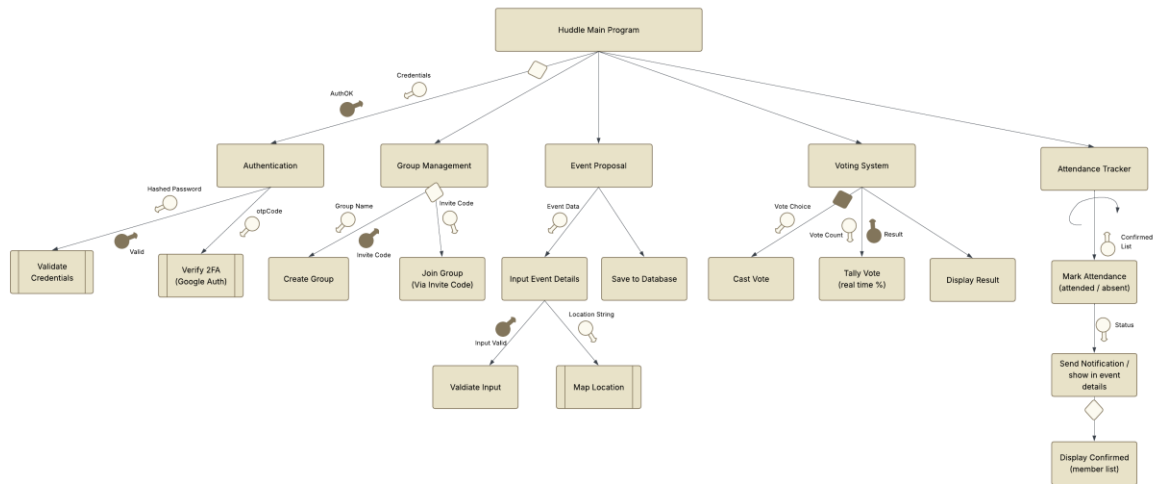
Level 0



Level 1 DFD: Breaks down internal processes — user management, group management, event handling, and voting logic.



Structure Chart: Shows the module hierarchy of the application from login through to event and attendance management.



2. Testing Team

The following team members will carry out UAT during Phase 4:

Name	Responsibilities
Michael Liondis	UAT Coordinator - designs test cases, oversees testing sessions, compiles results and final UAT report.
Mr Jones	Evaluator - reviews UAT plan and final results against project requirements.
Kevin Tran	End-user tester - performs usability and functional tests as a typical group member.
Ismail Rizvi	End-user tester - performs cross-device and cross-browser compatibility tests.
Kelvin Aung	End-user tester - performs security and group isolation tests using a separate test account.

3. Environmental Requirements

3.1 Hardware Requirements

Huddle must be tested across the following device types to satisfy the PWA responsiveness requirement (320px–1920px):

Device Type	Example	Purpose
Smartphone	iPhone or Android (any recent model)	Mobile responsiveness at ~320–430px width
Tablet (optional)	iPad or Android tablet	Mid-range screen testing at ~768px width
Laptop / Desktop	Any laptop or desktop PC/Mac	Full desktop layout at 1080px–1920px width

All devices require a stable internet connection to sync vote and event data.

3.2 Software Requirements

Huddle is a PWA - no installation is required. Testers must have the following:

Requirement	Details
Web Browser	Google Chrome (latest), Mozilla Firefox (latest), Safari (latest on iOS/macOS)
Google Authenticator	Installed on a smartphone (iOS App Store or Google Play) and linked to a Huddle test account.
Internet Connection	Active Wi-Fi or mobile data connection (4G minimum recommended).
Test Accounts	Minimum 2 separate Huddle accounts (to test group joining and multi-user voting simultaneously).

3.3 Network Requirements

- Stable internet connection (Wi-Fi or 4G/5G mobile data) on all test devices
- The Huddle backend server must be running and accessible at the test URL
- The SQL database must be live and connected to the backend during all test sessions

4. Test Scripts

The table below contains all UAT test cases. Each case maps to a specific functional or non-functional requirement. The 'Actual Result' and 'Pass/Fail' columns are to be completed during Phase 4 testing.

Note: UAT-015 (Role Assignment) is marked as a 'Nice to Have' feature - it should only be tested if the feature has been implemented.

Test ID	Requirement	Test Description	Expected Result	Actual Result	Pass/Fail
UAT-001	User Authentication (2FA)	Register a new account and complete Google Authenticator 2FA setup. Enter valid email/password, then enter the 6-digit TOTP code from Google Authenticator.	Account is created and user is logged in successfully. Invalid or expired codes are rejected with an error message.	(Phase 4)	—
UAT-002	User Login	Log in with an existing account. Enter correct email and password, then enter the current 6-digit 2FA code.	User is authenticated and redirected to the home/dashboard screen within 3 seconds.	(Phase 4)	—
UAT-003	Group Creation via Invite Code	Create a new friend group. Enter a group name and submit. Note the generated 6-digit alphanumeric invite code.	A new group is created and a unique 6-character invite code is displayed. The group appears in the user's group list.	(Phase 4)	—
UAT-004	Group Joining via Invite Code	Using a second test account, enter a valid invite code to join the group created in UAT-003.	The second user successfully joins the group and can see it in their group list. The group creator can see the new member.	(Phase 4)	—
UAT-005	Invalid Invite Code Rejection	Attempt to join a group using a made-up or expired invite	System displays a clear error message (e.g. 'Invalid or expired invite code').	(Phase 4)	—

		code (e.g. 'XXXXXX').	The user is NOT added to any group.		
UAT-006	Event Proposal System	As a group member, propose a new event. Enter an event name (max 30 chars), select a date/time, and enter a location (max 100 chars), then submit.	The event appears in the group's event feed and is visible to all group members. Input that exceeds character limits is rejected.	(Phase 4)	—
UAT-007	Voting System (Yes/No/Maybe)	As a group member, cast a 'Yes', 'No', and 'Maybe' vote on a proposed event. Then switch votes to verify the system updates correctly.	Votes are recorded and the real-time tally (percentage breakdown of Yes/No/Maybe) updates and is visible to all group members within 2 seconds.	(Phase 4)	—
UAT-008	Real-time Vote Update (Performance)	Have two test users open the same event simultaneously. User A casts a vote while User B watches the tally.	User B sees the updated vote percentage within 2 seconds of User A voting, without needing to refresh the page.	(Phase 4)	—
UAT-009	Attendance Tracker	After an event date has passed (or using a test past-dated event), the event proposer confirms attendance by marking who attended.	A 'Confirmed' list updates immediately upon marking. Attended/not-attended statuses are visible to all group members.	(Phase 4)	—
UAT-010	Group Data Isolation (Security)	Log in as a user who belongs to Group A only. Attempt to manually navigate to Group B's event URL (if known). Also verify that no Group B events appear in Group A's feed.	The user cannot view or interact with Group B's events. An access denied message or redirect occurs. Group A and B data is fully siloed.	(Phase 4)	—
UAT-011	Responsiveness – Mobile (320px)	Open the app on a smartphone (or resize browser to 320px wide).	All screens render correctly with no overlapping elements, cut-off	(Phase 4)	—

		Navigate through: login, group view, event proposal form, and vote tally screen.	text, or broken layouts. All buttons and inputs are fully usable.		
UAT-012	Responsiveness – Desktop (1920px)	Open the app on a desktop browser at full screen (1920px wide). Navigate through all core screens.	The layout scales correctly. Content is readable and well-spaced. No layout breaks or excessively stretched elements.	(Phase 4)	—
UAT-013	Usability – New User Onboarding	A first-time user (with no prior briefing) attempts to: create an account, create a group, and propose an event. Time the process.	The user completes all three actions in under 3 minutes without needing a tutorial or external help.	(Phase 4)	—
UAT-014	Cross-Browser Compatibility	Open the app in Chrome, Firefox, and Safari. Perform a login, vote on an event, and view the tally in each browser.	All features function identically across all three browsers with no visual or functional differences.	(Phase 4)	—
UAT-015	Role Assignment (Nice to Have)	On a confirmed event, assign a role to an attending member (e.g. 'Food Bringer'). Enter a custom role (min 3, max 50 characters) and also test selecting from the pre-defined dropdown.	The role is saved and displayed next to the member's name. Input outside 3–50 character limits is rejected. Profanity in custom roles is blocked.	(Phase 4)	—

All 'Actual Result' and 'Pass/Fail' fields above are to be completed during Phase 4 testing.